```
class Camera                                    <-- profit, revenue, cost, and
  constructor: (@price) ->                          price instance variables
  calculateRevenue: ->
    @revenue = (50 + (20 / 10) * (200 - @price)) * @price
  calculateCost: ->
    @cost = 140 + (100 * (50 + ((20 / 10) * (200 - @price))))
  calculateProfit: ->
    @calculateRevenue()
    @calculateCost()
    @profit = @revenue - @cost

phototaka500 = new Camera 162
phototaka500.calculateProfit()
console.log phototaka500.profit
# 7672
```

These two solutions produce the same result. The difference lies in the implementation. The first solution doesn't set any properties on the instance, whereas the second does. Consider for a minute which one is more like the following example that uses variables and shared state:

```
revenue = 0
cost = 0
sold = 0                    <-- Variables used to keep state
                                shared by the functions

calculateRevenue = (salePrice) ->
  revenue = sold * salePrice

calculateCost = (salePrice) ->
  cost = 140 + sold * 100

calculateNumberSold = (salePrice) ->
  sold = 50 + 20/10 * (200 - salePrice)

calculateProfit = (salePrice) ->
  calculateNumberSold salePrice
  calculateRevenue salePrice
  calculateCost salePrice
  revenue - cost
```

The object approach using *instance variables* is more like the version with shared state kept in variables. This style of programming is called *imperative programming*. All programs written in an imperative style have to answer the question of how to manage the explicit state. In an object-